Як насправді працюють Full Micro, MicroMacro та Structural A/U–E/U–E/E

Просте послідовне пояснення з однією мережею, формулами та схемами

Що треба зрозуміти з цього тексту

Усі варіанти нижче описують поширення інфекції в мережі, але вони **бачать різну кількість інформації про міжспільнотні контакти**. Full Micro бачить кожне ребро. Original MicroMacro бачить детальну мережу всередині кожної community, але між communities працює з агрегатами. Structural A/U, E/U та E/E — це окремий діагностичний simulator, де ми по одному повертаємо інформацію, втраченою агрегацією.

1. Почнемо з однієї й тієї самої мережі

Нехай є дві communities: A і B. Усередині кожної є звичайна контактна мережа людей. Між communities існують лише кілька реальних контактів. Червоні вузли вже заражені, сірі — susceptible. Синя рамка означає boundary/gateway-вузол, який має контакт з іншою community.

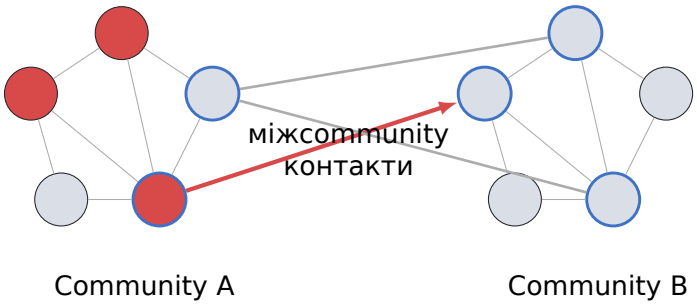


Figure 1: Одна фізична мережа. Зараз лише одне міжcommunity ребро є активним $I \to S$: від зараженого boundary-вузла A до susceptible boundary-вузла B.

Ця сама мережа буде подана всім симуляторам. Відмінність не в самій картинці, а в тому, **яку частину цієї картинки алгоритм використовує для міжcommunity передачі**.

2. Full Micro: алгоритм бачить усе

Full Micro — це повний Gillespie на всій мережі. Для нього немає принципової різниці між “внутрішнім” і “міжcommunity” ребром. Усі контакти — це звичайні ребра одного графа.

Для кожного ребра (u, v), де u infected, а v susceptible, існує канал зараження з rate

$$a_{uv} = \beta w_{uv} \mathbb{I}[X_u = I,\, X_v = S].$$

Якщо ребра невагові, $w_{uv} = 1$. Загальний infection hazard — це сума rates усіх активних $I \to S$ ребер:

$$\Theta_{\text{micro}} = \sum_{(u,v)\in E} a_{uv}.$$

Тобто Full Micro буквально запитує: *"Які конкретно заражені люди зараз мають контакт із конкретними susceptible людьми?"*

У мережі на рис. 1 між A і B є три фізичні ребра, але активне зараз тільки одне. Якщо всі мають rate β, точний міжcommunity hazard дорівнює

$$\lambda_{AB}^{\text{exact}} = \beta.$$

Коли це ребро спрацьовує, заражається **саме той susceptible-вузол, який стоїть на його кінці**. Full Micro не вигадує target: його визначає реальне ребро.

Інтуїція

Full Micro = "я знаю стан кожної людини, кожне ребро і тому точно знаю, які переходи зараз можливі".

Це відповідає твоєму попередньому exact Gillespie-підходу на contact network: інтенсивність susceptible-вузла визначається його зараженими сусідами, а наступний event вибирається серед реально доступних transitions.

3. Original MicroMacro: детально всередині, грубо між communities

MicroMacro робить головне спрощення: **усередині community він усе ще запускає локальний Gillespie на конкретних вузлах**, але між communities стискає інформацію.

Для macro-рівня community A перетворюється приблизно на такий запис:

$$A: \quad I_A,\ S_A,\ N_A,$$

а пара communities A, B має агреговану вагу W_{AB}, яка описує силу їхнього зв'язку.

У твоїй MoMLeT-моделі міжcommunity rate записується як

$$\lambda_{AB}(t) = \beta\, W_{AB}\, C_A(t)\, S_B(t)\, T,$$

де C_A — агрегована contagiousness source-community, а S_B — агрегована susceptibility destination-community. Для інтуїції це можна читати як схему

$$\lambda_{AB}^{A/U} \propto \beta W_{AB} \frac{I_A}{N_A} \frac{S_B}{N_B}.$$

У нашому toy example $W_{AB} = 3$ (три міжcommunity ребра), $I_A/N_A = 2/5$, а $S_B/N_B = 1$. Тоді агрегована формула може дати hazard близько

$$\lambda_{AB}^{A/U} = 3\beta \cdot \frac{2}{5} = 1.2\beta,$$



Macro не знає, **які саме** з двох infected у A стоять на boundary.

Macro не знає, **який саме** susceptible у B є endpoint реального cross-edge.

Figure 2: Так macro-рівень “бачить” ту саму мережу: не конкретні cross-edge states, а агреговані counts та вагу між communities.

хоча Full Micro бачить лише одне реально активне ребро і дає β.

Це і є **структурна апроксимація**: macro-рівень припускає, що infected і susceptible “розмішані” відносно boundary приблизно так само, як у всій community.

Коли macro-event відбувся, original MicroMacro не знає точний endpoint cross-edge. Тому він додає infection у **випадковий susceptible-вузол** destination-community.

3.1. Ще одна різниця: як MicroMacro рухає час

Original MicroMacro не просто має aggregate hazard. Він має власний production scheduler. Micro communities просуваються блоками довжини τ_{micro}, а macro-рівень накопичує hazard

$$H(t) = \int_{t_0}^{t} \Theta(s)\, ds$$

до випадкового порогу $E = -\ln U$. У статті цей інтеграл апроксимується на micro-grid, зокрема через midpoint rate в кожному інтервалі.

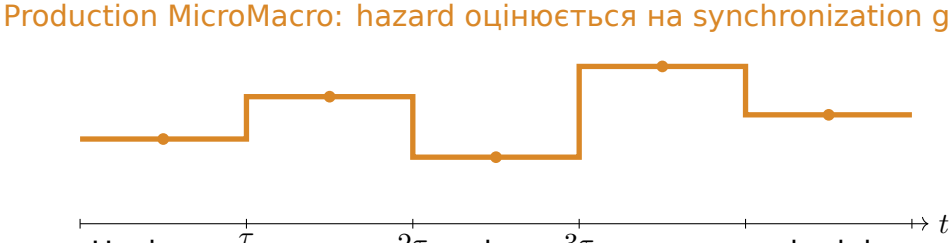


Figure 3: Схематично: MicroMacro має τ-синхронізацію та cumulative-hazard timing.

4. Structural A/U: ті самі грубі припущення, але інший simulator

Оце найважливіший момент.

Structural A/U означає:

$$\boxed{A = \text{Aggregate hazard}}, \qquad \boxed{U = \text{Uniform placement}}.$$

Тобто за **структурним змістом** A/U робить те саме грубе припущення, що й original MicroMacro:

- міжcommunity hazard рахується з агрегованої інформації, а не зі стану конкретних cross-edges;
- після import заражається випадковий susceptible у destination-community.

Але A/U — **не той самий executable code**. У diagnostic implementation немає production τ-scheduler у тому самому вигляді. Там micro і macro події конкурують як event-driven stochastic clocks.

Тому різниця така:

	Original MicroMacro	**Structural A/U**
Macro information	aggregate	aggregate
Placement	uniform susceptible	uniform susceptible
Hazard idea	та сама A/U approximation	та сама A/U approximation
Time scheduler	production τ + hazard integral	окремий event-driven scheduler
Executable	original user-facing simulator	окремий diagnostic simulator

Отже відповідь на питання "*чи hazard рахується інакше?*" така:

Коротка відповідь

У A/U **не обов'язково інша structural formula hazard**; за змістом вона така сама aggregate approximation, як у MicroMacro. Головна відмінність A/U від original MicroMacro — **не formula, а simulation scheduler та implementation mechanics**. Саме тому A/U потрібен, щоб подивитися на "чисту" structural approximation без частини production implementation artifacts.

4.1. Як A/U насправді вибирає наступну подію

Код цього diagnostic simulator є на Google Drive у файлі `two_layer_ctmc/experiments/hazard_placem`
У ньому немає однієї спільної "черги", де наперед лежать усі майбутні micro- і macro-події. Натомість на кожному стані алгоритм робить три прості кроки.

Крок 1. Він рахує сумарний rate усіх micro-подій і сумарний rate усіх macro-подій. Micro-події — це локальні infection/recovery events на внутрішніх ребрах усіх communities. Macro-події — це міжcommunity imports.

```
micro_channels, micro_weights = self._micro_channels(state)
micro_total = float(sum(micro_weights))
...
macro_total = float(sum(selected_hazards.values()))
```

Тобто якщо зараз усі можливі локальні events разом мають rate Θ_μ, а всі міжcommunity events разом — rate Θ_M, далі запускаються **два незалежні експоненційні годинники**:

```
dt_micro = self.micro_rng.expovariate(micro_total) \
    if micro_total > 0.0 else math.inf
```

```
dt_macro = self.macro_budget_rng.expovariate(macro_total) \
    if macro_total > 0.0 else math.inf
```

Математично це

$$T_\mu \sim \mathrm{Exp}(\Theta_\mu), \qquad T_M \sim \mathrm{Exp}(\Theta_M).$$

Крок 2. Алгоритм порівнює ці два часи. Хто менший — той тип події й відбувається першим:

```
if dt_micro < dt_macro:
    # execute a micro event
else:
    # execute a macro event
```

Наприклад, нехай сумарний micro-rate дорівнює 4, а macro-rate дорівнює 1. В одному конкретному кроці можуть випасти

$$T_\mu = 0.12, \qquad T_M = 0.37.$$

Тоді через 0.12 одиниці часу стається micro-event. Macro-event, який "планувався" на 0.37, не зберігається як готова подія: після зміни стану rates перераховуються і обидва годинники семплюються заново.

Крок 3. Після того як визначено тип події, всередині цього типу є своя roulette. Якщо виграв micro-clock, вибирається конкретний micro-channel пропорційно його rate:

```
event = _weighted_choice(
    micro_channels, micro_weights, self.micro_rng
)
```

Якщо виграв macro-clock, спочатку вибирається конкретна пара communities (i, j) пропорційно її hazard:

```
pairs = list(selected_hazards)
pair = _weighted_choice(
    pairs,
    [selected_hazards[p] for p in pairs],
    self.macro_pair_rng,
)
```

Після цього окремо вибирається target-вузол. Для A/U усі susceptible nodes destination-community мають однакову вагу 1:

```
target_weights = {
    node: 1.0
    for node in self.index.communities[destination]
    if state[node] == SUSCEPTIBLE
}
target = _weighted_choice(
    list(target_weights),
    list(target_weights.values()),
    self.placement_rng,
)
```

Отже правильна інтуїція така:

> **Не одна глобальна queue і не дві незалежні "рулетки подій".** Спочатку є **два clocks**: один для всього micro-блоку, другий для всього macro-блоку. Менший clock визначає тип наступної події. Уже після цього запускається roulette всередині переможного типу.

4.2. Чи “збивається” програвший clock? Чому це не помилка

На перший погляд здається, що якщо micro-event стався раніше, ми “викинули” вже sampled macro-time і тим самим змінили процес. Для експоненційних waiting times це не проблема через memoryless property.

Якщо

$$T_\mu \sim \mathsf{Exp}(\Theta_\mu), \qquad T_M \sim \mathsf{Exp}(\Theta_M),$$

то мінімум двох clocks має розподіл

$$\min(T_\mu, T_M) \sim \mathsf{Exp}(\Theta_\mu + \Theta_M),$$

а ймовірність, що першим буде micro-event,

$$P(T_\mu < T_M) = \frac{\Theta_\mu}{\Theta_\mu + \Theta_M}.$$

Якщо після цього всередині micro-блоку channel r має rate a_r, то

$$P(r \text{ є наступною подією}) = \frac{\Theta_\mu}{\Theta_\mu + \Theta_M} \cdot \frac{a_r}{\Theta_\mu} = \frac{a_r}{\Theta_\mu + \Theta_M}.$$

Це рівно та сама ймовірність, яку дала б **одна велика Gillespie roulette**, де всі micro- і macro-channels були б складені в один список.

Тому для стану, в якому rates не змінюються між подіями, схема

$$\boxed{\text{два competing exponential clocks + conditional roulette}}$$

статистично еквівалентна схемі

$$\boxed{\text{один global Gillespie clock + одна global roulette}}.$$

Коли один event відбувся, стан мережі змінюється, отже змінюються і rates. Тоді все одно треба перебудувати майбутні hazards. Саме тому повторний sampling після кожної події є природним.

Важлива межа цього аргументу: він працює завдяки експоненційному waiting-time law. Для неекспоненційних clocks просто викидати залишковий час було б уже некоректно.

4.3. Чим це відрізняється від production MicroMacro

Original production MicroMacro на Drive має іншу логіку часу у `two_layer_ctmc/orchestrators/micro`. Він не семплює на кожному стані пару T_μ, T_M і не вибирає їх мінімум. Macro-рівень натомість має один exponential hazard-budget:

```
thresh_int = -math.log(random.random())
int_accum = 0.0
```

Далі час рухається до grid-boundary $t + \tau_{\text{micro}}$, а macro-hazard оцінюється в midpoint за допомогою micro snapshots:

```
t_grid = min(t + self.tau_micro, self.T_end)
t_mid = t + 0.5 * (t_grid - t)
...
total_hazard_mid = self.macro.total_hazard_given(I_mid, S_mid)
```

Якщо накопичений rectangle hazard перетнув budget, macro-event ставиться всередині цього інтервалу:

```
if int_accum + total_hazard_mid * dt >= thresh_int:
    t_event = t + (thresh_int - int_accum) / total_hazard_mid
```

До цього t_{event} кожен micro-model симулює свої внутрішні Gillespie events, після чого вже вибирається macro-pair:

```
i, j = self.macro.sample_transfer()
```

Отже production MicroMacro і A/U відрізняються не лише стилем коду. У них різна організація часу:

	Production MicroMacro	**Diagnostic A/U**
Micro timing	окремі micro-models просуваються до заданого calendar time	один сумарний micro clock для всіх internal channels
Macro timing	cumulative hazard budget на τ-grid	один exponential macro clock у поточному state
Micro vs macro	взаємодіють через synchronization/integration logic	безпосередньо конкурують два clocks
Вибір macro pair	roulette після macro threshold crossing	roulette після перемоги macro clock
Placement A/U	uniform susceptible	uniform susceptible

Саме тому A/U корисний як “чистий” diagnostic scheduler, але він не є точним executable equivalent original MicroMacro.

5. E/U: виправляємо тільки hazard

Тепер ми залишаємо uniform placement, але повертаємо точну інформацію про реальні міжcommunity ребра.

Для pair $A \to B$ exact hazard:

$$\lambda_{AB}^{E/U} = \beta \sum_{(u,v)\in E_{AB}} w_{uv}\, \mathbb{I}[X_u = I,\, X_v = S].$$

У toy network з рис. 1 активне тільки одне cross-edge, тому

$$\lambda_{AB}^{E/U} = \beta,$$

а не 1.2β, як у нашому простому aggregate прикладі.

Але після того, як macro-event відбувся, E/U все ще робить “U”: вибирає **будь-який susceptible** у B, а не обов’язково endpoint активного ребра.

Тому перехід A/U $\to$ E/U вимірює насамперед помилку від **aggregate hazard**.

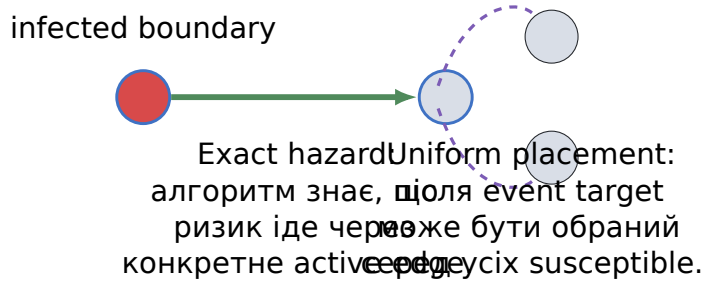


Figure 4: E/U знає правильно “скільки ризику”, але ще не обов’язково правильно “куди саме” має прийти infection.

6. E/E: виправляємо і hazard, і placement

E/E означає:

$$\boxed{E = \text{Exact hazard}}, \qquad \boxed{E = \text{Exact placement}}.$$

Hazard такий самий exact active-edge sum, як в E/U. Але тепер, якщо event вибрано через конкретне ребро (u, v), заражається саме його endpoint v.

Тобто E/E знає обидві речі:

1. **як часто** має відбуватися inter-community transmission;
2. **куди саме** він може потрапити.

Перехід E/U $\rightarrow$ E/E ізолює помилку від **uniform placement**.

7. То яка різниця між E/E і Full Micro?

Це тонке, але просте питання.

Full Micro має одну глобальну мережу. Для нього internal та cross-community edges — звичайні ребра одного CTMC. Ніякого спеціального macro-рівня для фізики зараження йому не потрібно.

E/E усе ще написаний як двошаровий diagnostic simulator: internal events і inter-community events організовані окремими блоками логіки. Але для inter-community частини він уже використовує точні active edges та exact endpoint.



Figure 5: E/E може бути математично еквівалентним Full Micro, хоча кодова архітектура інша.

Якщо одночасно виконуються умови:

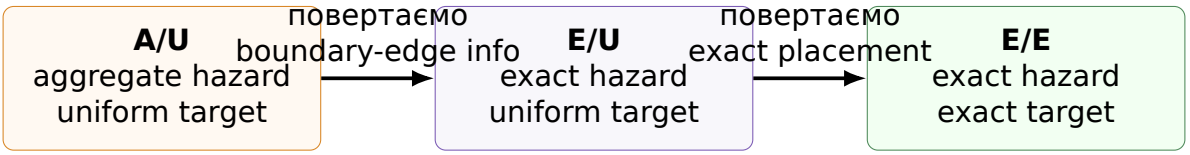
- internal Gillespie rules однакові;
- exact cross-edge rates однакові;
- recovery та інші event types однакові;

- exact target визначається тим самим active edge;

то обидва algorithms реалізують той самий CTMC generator. Тоді вони не зобов'язані дати однакову конкретну random trajectory, але повинні давати **той самий статистичний розподіл** при багатьох replications.

І саме тому E/E — хороший sanity check. Якщо E/E не збігається з Full Micro статистично, проблема вже не в A/U approximation, а в тому, що diagnostic implementation не відтворює exact process.

8. Уся логіка A/U → E/U → E/E одним ланцюжком



A/U → E/U показує, скільки похибки дає aggregate hazard.
E/U → E/E показує, скільки похибки дає uniform placement.

Figure 6: Structural ablation ladder: по одному повертаємо інформацію, яку macro abstraction втратила.

9. Що вважати основним результатом для статті

Тут важливе методологічне правило.

Якщо ми хочемо сказати щось саме про **твій user-facing MicroMacro method**, primary experiments краще робити на original 'MicroMacroSimulator' і змінювати мінімально необхідну деталь:

Experiment	**Найчистіша реалізація**
A/U baseline	original MicroMacro без змін
E/U	original MicroMacro + замінити лише macro hazard provider на exact active-edge hazard
E/E	те саме + замінити лише target selector на exact edge endpoint
Gateway near/random/far	simulator взагалі не змінювати; змінювати тільки topology
$d \times k$ grid	simulator не змінювати; змінювати лише параметри мережі

Окремий 'HazardPlacementSimulator' все одно корисний: він дуже добре підходить як **secondary mechanistic diagnostic**, бо прибирає production scheduler artifacts і дозволяє чисто розкласти structural error. Але його результати краще подавати як підтвердження механізму, а не як єдине свідчення того, що отримає користувач original MicroMacro.

10. Найкоротше резюме

Full Micro: бачить кожен вузол і кожне active edge. Exact hazard, exact target.

Original MicroMacro: усередині community бачить nodes, між communities бачить aggregate I, S, W. Aggregate hazard, uniform target, production τ-scheduler.

Structural A/U: ті самі aggregate/uniform structural assumptions, але окремий event-driven diagnostic simulator.

E/U: повертає exact cross-edge hazard, але target ще uniform.

E/E: повертає і exact hazard, і exact endpoint. Якщо решта event rules збігаються з Full Micro, статистично має відтворювати той самий CTMC.

Примітка про джерела

Опис original MicroMacro, його двох рівнів, macro-rate та cumulative-hazard synchronization базується на наданій роботі *Towards complexity reduction of large-scale epidemic simulation in two-scale networks* (MoMLeT-2025). Опис Full Micro спирається на надану роботу *Simulating epidemic peak dynamics on complex networks using efficient Gillespie algorithms*. Розрізнення Structural A/U–E/U–E/E як окремої diagnostic implementation базується на перевіреному source code експериментів у Google Drive в межах поточного аудиту.